# Computer Assignment

Graded

**Student**

Ioannis Alexandrides

**Total Points**

**40 / 40 pts**

**Autograder Score**

**0.0 / 0.0**

**Question 2**

## Task 1

**13** / 13 pts

Task 1a

✔ **+ 3 pts** Task 1a) correct

**+ 0 pts** Task 1a) not attempted

---

Task 1b

✔ **+ 2 pts** Task 1b) correct

**+ 0 pts** Task 1b) not attempted

---

Task 1c

✔ **+ 3 pts** Task 1c) correct

**+ 0 pts** Task 1c) not attempted

---

Task 1d

✔ **+ 5 pts** Task 1d) correct

**+ 0 pts** Task 1d) not attempted

Question 3

## Task 2

**27** / 27 pts

Task 2a

✔ **+ 8 pts** Task 2a) correct

**+ 0 pts** Task 2a) not attempted

---

Task 2b

✔ **+ 5 pts** Task 2b) correct

**+ 0 pts** Task 2b) not attempted

---

Task 2c

✔ **+ 7 pts** Task 2c) correct

**+ 0 pts** Task 2c) not attempted

---

Task 2d

✔ **+ 4 pts** Task 2d) correct

**+ 0 pts** Task 2d) not attempted

---

Task 2e

✔ **+ 3 pts** Task 2e) correct

**+ 0 pts** Task 2e) not attempted

## Autograder Results

This assignment does not have an autograder configured.

## Submitted Files

**computer_assignment_final.pdf** Download

Your browser does not support PDF previews. You can download the file instead.

# Numerical ODEs – Computer Assignment (Questions)


# Ioannis Alexandrides, s2767276


## Instructions

- Commenting your code is not required, but it is recommended, since we may use comments to judge whether to award partial credit for solutions that aren't fully correct.
- You should not need to use packages other than `numpy`, `scipy` and `matplotlib`.

## Note

- So far in the course you have learned about one type of stability, that in Definition 3.2.3 in the lecture notes. This definition (also commonly called zero stability) is important for proving convergence of numerical methods as $h \to 0$. However, in practice it is also common to describe numerical solutions as "numerically unstable", when, for a given step size $h > 0$, local errors accumulate and cause the numerical solution to diverge. We will say that a numerical solution is *numerically unstable* if it is unbounded, and stable otherwise. Numerically unstable solutions will typically contain `nan` or `inf` (or `-inf`) values when implemented in `numpy`.

## Task 1: Stiff scalar ODE (13 marks)

We study the Prothero-Robinson equation

$$\frac{dy}{dt} = \lambda(\sin t - y) + \cos t, \quad \lambda > 0.$$

With the initial condition $y(t_0) = \sin t_0$, the exact solution is

$$y(t) = \sin t$$

for all $t$, independently of $\lambda$.

However, for initial conditions $y(t_0) \neq \sin t_0$ and large $\lambda$, solutions quickly converge to $\sin t$ after $t_0$. This is due to the first term on the right-hand side -- notice that large $\lambda$ implies large $\frac{dy}{dt}(t_0)$ in this case. It is a challenge for numerical methods to capture this fast transient behaviour. Even when $y_0 = y(t_0) = \sin(t_0)$, numerical solutions can be affected by this $O(\lambda)$ term, since numerical errors will cause $y_n$ to differ from $\sin(t_n)$, for $n > 0$. We refer to this property of the ODE as *stiffness*.

Throughout Task 1 we assume the initial condition $y_0 = y(t_0) = 0$ at $t_0 = 0$
.

In [1]:

```
import warnings
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
%matplotlib inline
```

## Task 1a: Forward Euler, RHS, and exact solution (3 marks)

- Implement forward Euler as a function `euler_forward(T, f, y0, lam)` that returns the array of $y_n$ values for a uniform time grid `T` and ODE with RHS $f$. The input `y0` represents an arbitrary initial condition.
- Implement the RHS of the ODE as a function `stiff_eqn(t, y, lam)`.
- Implement a function `solution(T, lam)` that evaluates the exact solution $y(t) = \sin t$ for a uniform time grid `T`.

Note: we use `lam` to represent $\lambda$, since `lambda` is reserved for defining anonymous functions in Python.

In [2]:

```
def euler_forward(T, f, y0, lam):
    """
    Forward Euler method for a scalar ODE.
    Parameters:
        T   : array of time points (uniform grid assumed)
        f   : RHS function f(t, y, lam)
        y0  : initial condition
        lam : parameter lambda
    Returns:
        Y : array of numerical solution values at times T
    """
    # implement forward Euler
    import warnings
    N = len(T)
    Y = np.zeros(N)
    Y[0] = y0
    with warnings.catch_warnings():
        warnings.simplefilter('ignore', RuntimeWarning)
        for i in range(N - 1):
            h = T[i + 1] - T[i]
            Y[i + 1] = Y[i] + h * f(T[i], Y[i], lam)
            if not np.isfinite(Y[i + 1]):   # blow-up detected
                Y[i + 1:] = np.nan
                break
    return Y


def stiff_eqn(t, y, lam):
    # fill in dy/dt for the given ODE
    return lam * (np.sin(t) - y) + np.cos(t)


def solution(T, lam):
```

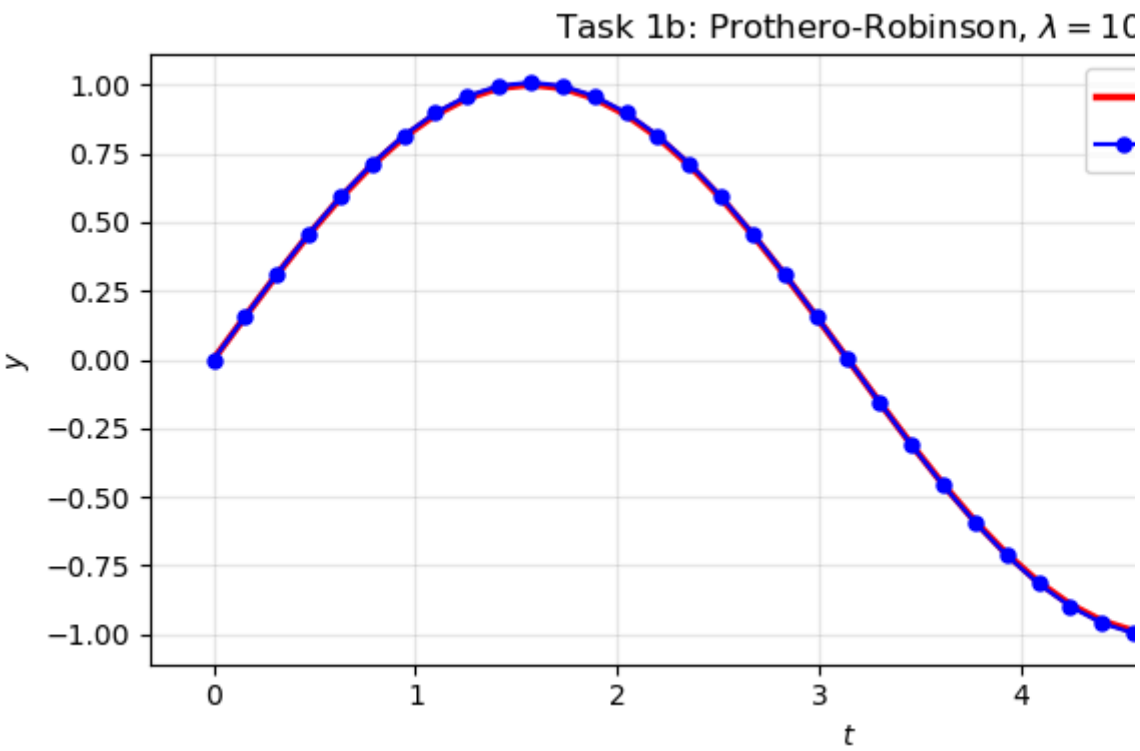
```
    # return exact solution values
    return np.sin(T)
```

## Task 1b: Plot comparison (2 marks)

With $\lambda = 10$, $t \in [0, 2\pi]$, and step size $h = \pi/20$, plot forward Euler and the exact solution together.

In [3]:

```
y0 = 0.0
lam = 10.0
t0, tf, h = 0.0, 2.0 * np.pi, np.pi / 20
T = np.arange(t0, tf + h, h)

# compute forward Euler and exact solution on T
Y_euler = euler_forward(T, stiff_eqn, y0, lam)
Y_exact = solution(T, lam)

# plot
plt.figure(figsize=(8, 4))
plt.plot(T, Y_exact, 'r-',  linewidth=2.5, label=r'Exact  $y(t) = \sin t$')
plt.plot(T, Y_euler, 'b-o', linewidth=1.5, markersize=5, label=r'Forward Euler
($h=\pi/20$)')
plt.xlabel('$t$')
plt.ylabel('$y$')
plt.title(f'Task 1b: Prothero-Robinson, $\\lambda={int(lam)}$')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f'Max |error| = {np.max(np.abs(Y_euler - Y_exact)):.4e}')
```



Max |error| = 7.8635e-03

## Task 1c: Backward Euler and numerical stability (3 marks)

Implement backward Euler specialised to this ODE (not for general $f$) as a function `euler_backward(T, y0, lam)`. Then with $\lambda = 10000$, compare forward and backward Euler with step size $h = \pi/20$; comment on the numerical stability of each method (with the specified $\lambda$ and $h$) in a markdown cell below.

**Derivation of the closed-form update.**

The backward Euler step requires solving $y_{n+1} = y_n + h\, f(t_{n+1}, y_{n+1}, \lambda)$. Substituting $f$:

$$y_{n+1} = y_n + h\big[\lambda(\sin t_{n+1} - y_{n+1}) + \cos t_{n+1}\big]$$

$$y_{n+1}(1 + h\lambda) = y_n + h\lambda \sin t_{n+1} + h \cos t_{n+1}$$

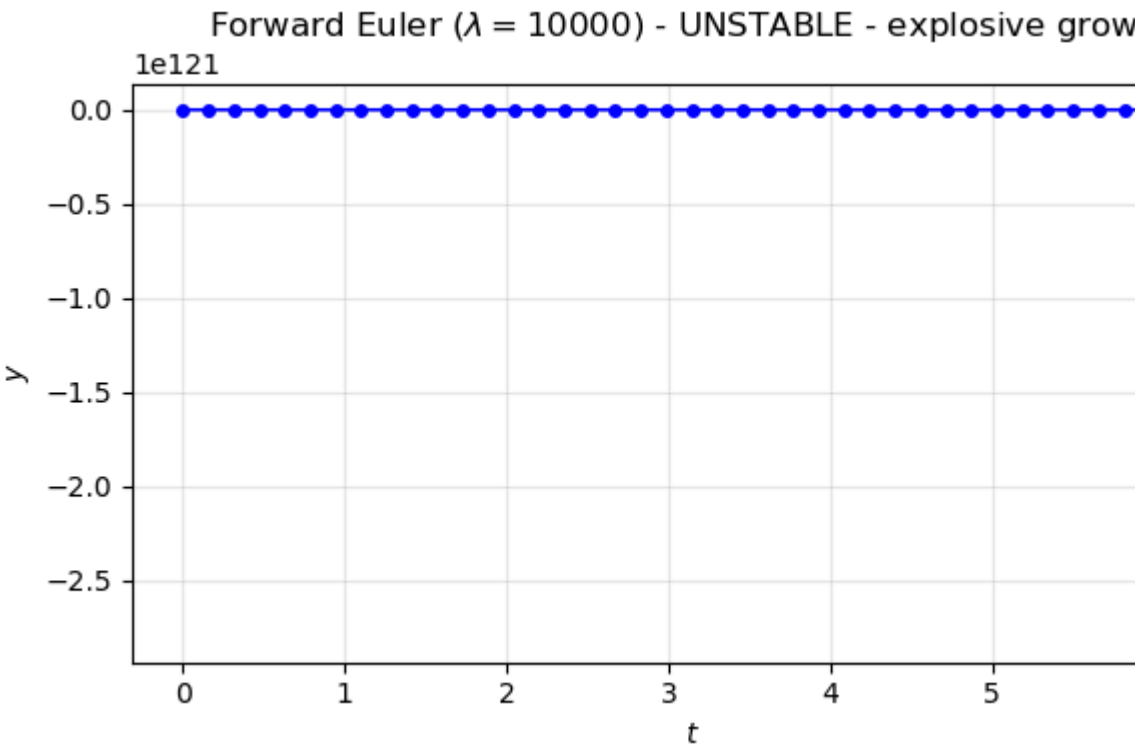
$$\boxed{y_{n+1} = \frac{y_n + h\lambda \sin t_{n+1} + h \cos t_{n+1}}{1 + h\lambda}}$$

The ODE is linear in $y$, so the implicit equation has an exact closed form with no iteration needed.

In [4]:

```
def euler_backward(T, y0, lam):
    """
    Backward Euler specialised to dy/dt = lam(sin t - y) + cos t

    Uses the derived closed-form update:
        y_{n+1} = (y_n + h*lam*sin(t_{n+1}) + h*cos(t_{n+1})) / (1 + h*lam).
    """
    # implement closed-form update for ODE
    N = len(T)
    Y = np.zeros(N)
    Y[0] = y0
    for i in range(N - 1):
        h = T[i + 1] - T[i]
        Y[i + 1] = (Y[i] + h * lam * np.sin(T[i + 1]) + h * np.cos(T[i + 1])) / (1.0 + h * lam)
    return Y


lam = 10000.0
t0, tf, h = 0.0, 2.0 * np.pi, np.pi / 20
T = np.arange(t0, tf + h, h)

# compute forward Euler, backward Euler and the exact solution on T
Y_fwd   = euler_forward(T, stiff_eqn, 0.0, lam)
Y_bwd   = euler_backward(T, 0.0, lam)
Y_exact = solution(T, lam)
```

```
overflow = not np.all(np.isfinite(Y_fwd))
explosive = np.max(np.abs(Y_fwd)) > 1e6

if overflow:
    status = 'UNSTABLE - overflow'
elif explosive:
    status = 'UNSTABLE - explosive growth'
else:
    status = 'Stable'

# plot results
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].plot(T, Y_fwd, 'b-o', markersize=4, linewidth=1.2)
axes[0].set_title(f'Forward Euler ($\\lambda={int(lam)}$) - {status}')
axes[0].set_xlabel('$t$'); axes[0].set_ylabel('$y$'); axes[0].grid(True, alpha=0.3)
axes[1].plot(T, Y_exact, 'r-',  linewidth=2.5, label='Exact')
axes[1].plot(T, Y_bwd,   'g-o', linewidth=1.5, markersize=5, label='Backward Euler')
axes[1].set_title(f'Backward Euler ($\\lambda={int(lam)}$) - STABLE')
axes[1].set_xlabel('$t$'); axes[1].set_ylabel('$y$')
axes[1].legend(); axes[1].grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

print(f'Forward Euler  - status : {status}')
print(f'Backward Euler - max |error| : {np.max(np.abs(Y_bwd - Y_exact)):.4e}')
```




```
Forward Euler  - status : UNSTABLE - explosive growth
Backward Euler - max |error| : 7.8377e-06
```

**Comment on numerical stability:**

For this problem the stiffness comes from the term involving $\lambda$, and that's what controls the stability behaviour of the methods.

For forward Euler, the amplification factor is

$$1 - h\lambda.$$

For stability we need

$$|1 - h\lambda| \leq 1.$$

This gives the condition

$$h \leq \frac{2}{\lambda}.$$

With $\lambda = 10000$ and $h = \frac{\pi}{20} \approx 0.157$, we get

$$h\lambda \approx 1571,$$

which is far larger than 2. This means numerical errors are amplified massively at every step. Even if the solution doesn't immediately become NaN, it clearly grows in an unrealistic way and is numerically unstable.

For backward Euler, the amplification factor is

$$\frac{1}{1 + h\lambda}.$$

For all $h > 0$ and $\lambda > 0$, we have

$$\left| \frac{1}{1 + h\lambda} \right| < 1,$$

so the method is unconditionally stable.

This matches what we see in the plots: forward Euler struggles badly for large $\lambda$, while backward Euler remains stable and tracks $\sin t$ very closely. This clearly shows why implicit methods are much better suited for stiff problems.

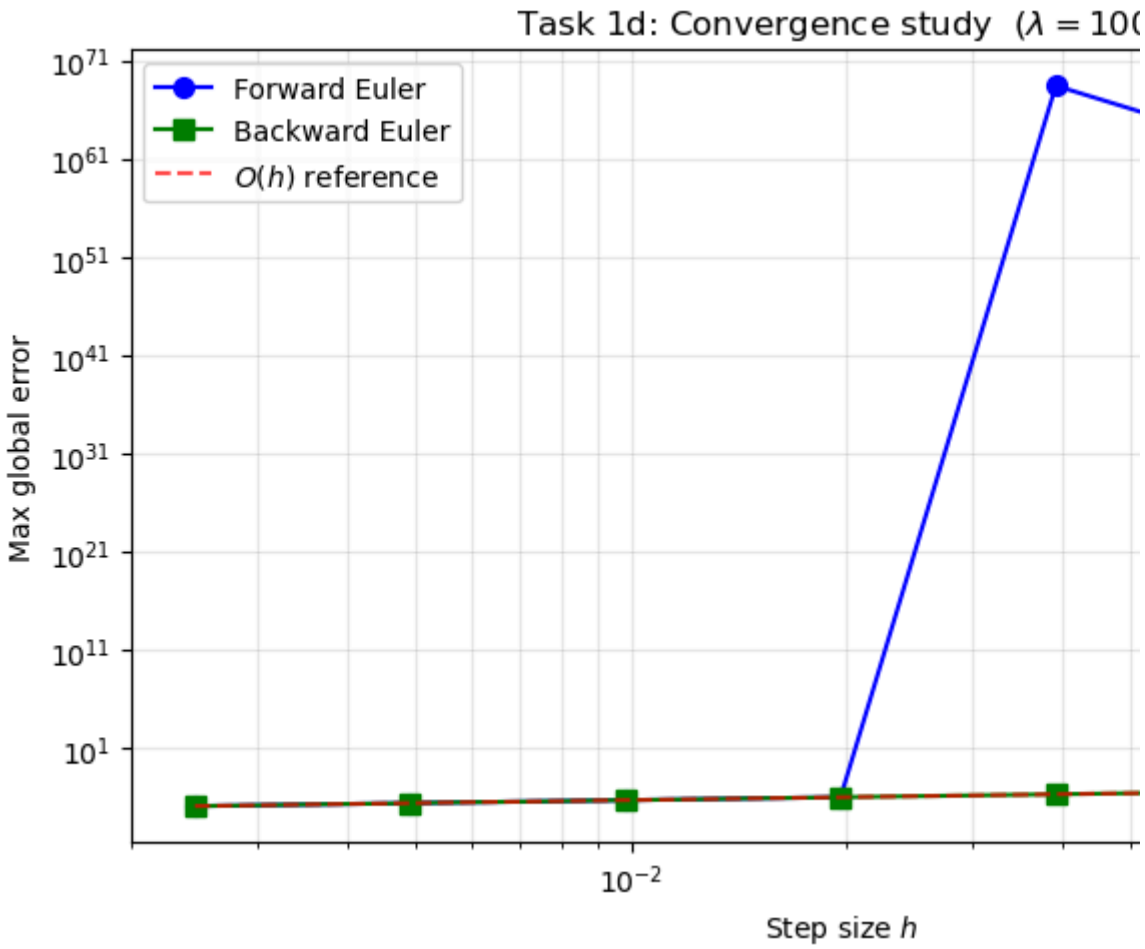
## Task 1d: Error study (5 marks)

- Using $\lambda = 100$ and $h \in \{2\pi/N : N \in \{40, 80, 160, 320, 640, 1280, 2560\}\}$ on $t \in [0, 2\pi]$, compute the maximum absolute value of the global error for forward and backward Euler: $\max_{n=0}^{N} |y_n - y(t_n)|$. Print the results as a table.
- By considering the ratios between these errors, roughly estimate the order of convergence of backward Euler. Briefly compare the performance of the two methods on this problem.

In [5]:

```
y0 = 0.0
lam = 100.0
t0, tf = 0.0, 2.0 * np.pi
h = 2 * np.pi / np.array([40, 80, 160, 320, 640, 1280, 2560])
error_forw = []
error_back = []

for i, _ in enumerate(h):
    T = np.arange(t0, tf + h[i], h[i])
    # compute forward Euler, backward Euler, exact solution arrays on T
    Y_fwd   = euler_forward(T, stiff_eqn, y0, lam)
    Y_bwd   = euler_backward(T, y0, lam)
    Y_exact = solution(T, lam)
    # append maximum absolute value of global errors to error_forw and
error_back
    error_forw.append(np.max(np.abs(Y_fwd - Y_exact)))
    error_back.append(np.max(np.abs(Y_bwd - Y_exact)))

error_forw = np.array(error_forw)
error_back = np.array(error_back)
N_values   = np.array([40, 80, 160, 320, 640, 1280, 2560])
h_values   = 2 * np.pi / N_values

# estimate order of convergence for backward Euler (fit log-log)
p_back = np.polyfit(np.log(h_values), np.log(error_back), 1)[0]
stable = np.isfinite(error_forw)

# create and display a small table of N, backward error, forward error
print(f"{'N':>6}  {'h':>9}  {'Fwd Euler Error':>18}  {'Ratio':>7}  {'Bwd Euler Error':>18}
{'Ratio':>7}")
print('-' * 78)
for i, N in enumerate(N_values):
    rf = error_forw[i-1]/error_forw[i] if (i>0 and np.isfinite(error_forw[i-1]) and
np.isfinite(error_forw[i])) else float('nan')
    rb = error_back[i-1]/error_back[i] if i > 0 else float('nan')
    fe = f'{error_forw[i]:.4e}' if np.isfinite(error_forw[i]) else '     UNSTABLE'
    print(f"{N:>6}  {h_values[i]:>9.5f}  {fe:>18}  {rf:>7.4f}  {error_back[i]:>18.4e}
{rb:>7.4f}")

print(f'\nEstimated order -- Backward Euler : {p_back:.3f}  (expected: 1.0)')
print(f'Stability threshold: h* = 2/lam = {2/lam:.5f}')

plt.figure(figsize=(8, 5))
if stable.any():
    plt.loglog(h_values[stable], error_forw[stable], 'bo-', lw=1.5, ms=7, label='Forward
Euler')
plt.loglog(h_values, error_back, 'gs-', lw=1.5, ms=7, label='Backward Euler')
h_ref = np.array([h_values.min(), h_values.max()])
plt.loglog(h_ref, error_back[0] * (h_ref / h_values[0]), 'r--', alpha=0.7, label=r'$O(h)$
reference')
plt.xlabel('Step size $h$')
plt.ylabel(r'Max global error')
plt.title(f'Task 1d: Convergence study  ($\\lambda={int(lam)}$)')
plt.legend(); plt.grid(True, alpha=0.3, which='both'); plt.tight_layout(); plt.show()
```

```
   N        h     Fwd Euler Error    Ratio     Bwd Euler Error    Ratio
--------------------------------------------------------------------------------
  40    0.15708        1.6762e+42      nan        7.8268e-04      nan
  80    0.07854        4.7719e+61   0.0000        3.9220e-04   1.9956
 160    0.03927        2.5738e+68   0.0000        1.9627e-04   1.9983
 320    0.01963        9.8195e-05  2621062417550746112953291863834949458516408 32
 640    0.00982        4.9087e-05   2.0004        4.9082e-05   1.9999
1280    0.00491        2.4543e-05   2.0000        2.4542e-05   1.9999
2560    0.00245        1.2271e-05   2.0000        1.2271e-05   2.0000

Estimated order -- Backward Euler : 0.999  (expected: 1.0)
Stability threshold: h* = 2/lam = 0.02000
```




**Discussion:**

**Backward Euler** behaves exactly as theory predicts. Each time we halve $h$, the error is roughly halved as well. The error ratios are close to 2, which confirms first-order convergence.

The fitted slope from the log–log plot is approximately

$$p \approx 1,$$

which matches the expected $O(h)$ global error.

**Forward Euler** is numerically unstable for the coarser grids. With $\lambda = 100$ the stability condition requires $h < 2/\lambda = 0.02$. For small $N$ this is violated, producing overflow. Only once $h$ drops below the threshold do finite results appear, at the same first-order rate.

**Key takeaway:** the step size restriction is driven by stability, not accuracy. So for stiff problems, explicit methods are inefficient because we are forced to take very small $h$ just to remain stable. Implicit methods like backward Euler avoid this restriction and are therefore much more practical.

## Task 2: Van der Pol oscillator (27 marks)

The Van der Pol oscillator is a classic nonlinear ODE arising in electrical circuits and other systems. It exhibits stiffness for large values of the parameter $\mu > 0$. In this task you will use the `scipy` function `solve_ivp` as well as your own solvers.

### Using `solve_ivp` (and what it computes)

- By default `solve_ivp` uses the `RK45` method. This is a numerical method which adapts the step size used as the ODE is solved. It does this by applying two distinct Runge-Kutta methods (one which is fourth order and one which is fifth order) at each time step and using the difference between these two approximations to estimate the approximation error that is being made with the current step size. If the error is (estimated to be) too large, the step size is made smaller -- if the error is smaller than necessary, the step size is made larger. Instead of specifying a step size, users specify error tolerance using the arguments `rtol` and `atol`.
- Interface: `solve_ivp(f, [t0, tf], y0, rtol=1e-8, atol=1e-10)`; for systems of ODEs, `y0` is a vector (e.g., `np.array([2.0, 0.0])`).
- The RHS function `f` should have the signature `f(t, y)` and return an array. If your equations depend on a parameter (like $\mu$), you can use a lambda function: `lambda t, y: f(t, y, mu)`.
- See the SciPy documentation for more information – [https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html](https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html).

### Background

The Van der Pol oscillator evolves according to

$$\frac{d^2x}{dt^2} - \mu(1 - x^2)\frac{dx}{dt} + x = 0.$$

For small $\mu$ solutions are almost sinusoidal, but for larger $\mu$ the equation becomes stiff.

### Task 2a: Mild stiffness (8 marks)

1) Write the Van der Pol oscillator as a first-order system in terms of variables $y_1 = x$ and $y_2 = dx/dt$. 2) Complete the implementation of that

system in the code cell below (`vdp_eqs`). 3) With $\mu = 2$, $y_1(0) = 2$, $y_2(0) = 0$, simulate to $t_f = 20$ using `solve_ivp`; plot $y_1(t)$, $y_2(t)$, and the phase plane ($y_1$ vs $y_2$). 4) Report: (a) the approximate period between peaks of $y_1$; (b) the maximum and minimum of $y_1$ attained on $[0, 20]$.

Hint: For part 4.(a) you could write code to identify the times at which $y_1$ changes sign.

**Answer to 1):** Setting $y_1 = x$ and $y_2 = dx/dt$ and rearranging:

$$\frac{dy_1}{dt} = y_2, \qquad \frac{dy_2}{dt} = \mu(1 - y_1^2)\, y_2 - y_1$$

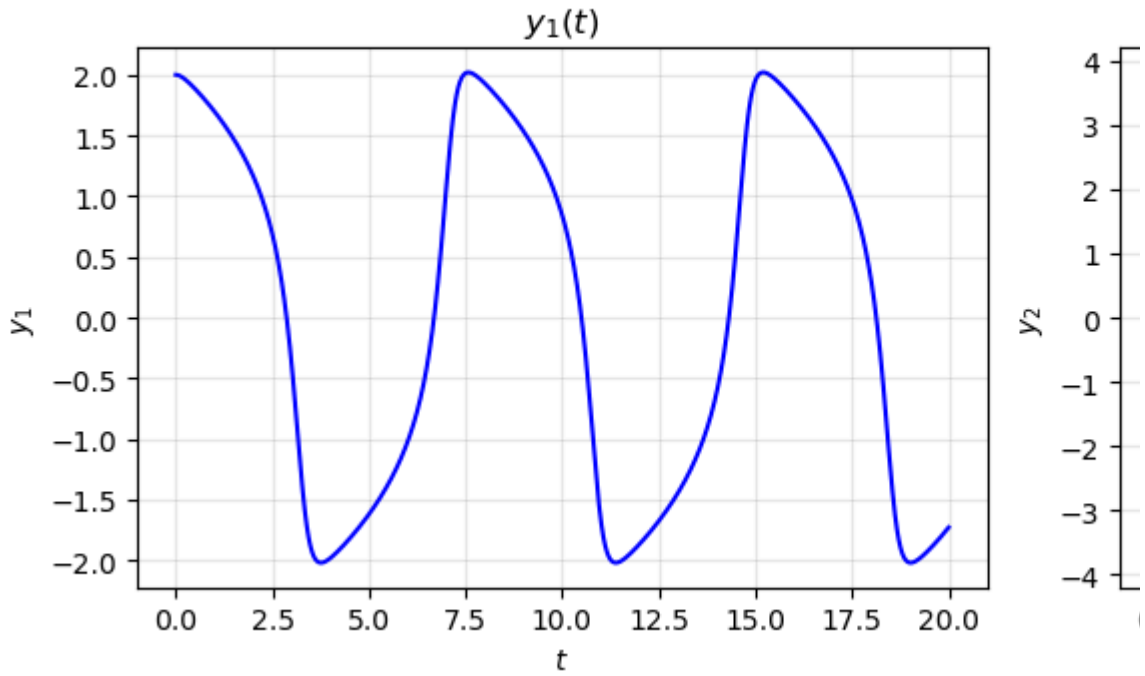
In [6]:

```
# Van der Pol RHS for use with solve_ivp

# Implement the first-order system you stated above (y1 = x, y2 = dx/dt).
def vdp_eqs(t, y, mu):
    """
    Van der Pol oscillator written as a first-order system.
    Let:
        y1 = x
        y2 = dx/dt
    Then:
        dy1/dt = y2
        dy2/dt = mu (1 - y1^2) y2 - y1
    """
    y1, y2 = y
    dydt = np.empty_like(y)
    # fill in dy1/dt and dy2/dt using your stated equations
    dydt[0] = y2
    dydt[1] = mu * (1.0 - y1**2) * y2 - y1
    return dydt
```

In [7]:

```
mu = 2.0
y0 = np.array([2.0, 0.0])
tf = 20.0

# solve using solve_ivp to T=20
sol = solve_ivp(lambda t, y: vdp_eqs(t, y, mu), [0.0, tf], y0,
                rtol=1e-8, atol=1e-10, dense_output=True)

# plot y1(t) and y2(t) and phase plane
t_plot = np.linspace(0.0, tf, 3000)
y_plot = sol.sol(t_plot)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].plot(t_plot, y_plot[0], 'b-', lw=1.5)
axes[0].set(xlabel='$t$', ylabel='$y_1$', title='$y_1(t)$'); axes[0].grid(True, alpha=0.3)
axes[1].plot(t_plot, y_plot[1], 'r-', lw=1.5)
axes[1].set(xlabel='$t$', ylabel='$y_2$', title='$y_2(t)$'); axes[1].grid(True, alpha=0.3)
axes[2].plot(y_plot[0], y_plot[1], 'g-', lw=1.2)
```

```
axes[2].set(xlabel='$y_1$', ylabel='$y_2$', title='Phase plane ($y_1$ vs $y_2$)');
axes[2].grid(True, alpha=0.3)
plt.suptitle(f'Task 2a: Van der Pol oscillator ($\\mu={mu}$)', fontsize=12)
plt.tight_layout(); plt.show()

# compute quantities
# 4(a): period via rising zero-crossings of y1, 4(b) is just max and min of y1 on [0,
20]
y1 = y_plot[0]
crossing_times = []
for i in range(len(y1) - 1):
    if y1[i] < 0.0 and y1[i + 1] >= 0.0:
        t_c = t_plot[i] + (-y1[i]) / (y1[i+1] - y1[i]) * (t_plot[i+1] - t_plot[i])
        crossing_times.append(t_c)

period = np.mean(np.diff(crossing_times))
print(f'4(a) Approximate period = {period:.4f} time units')
print(f'4(b) Max of y1 on [0, {int(tf)}] = {np.max(y1):+.6f}')
print(f'     Min of y1 on [0, {int(tf)}] = {np.min(y1):+.6f}')
```




```
4(a) Approximate period = 7.6299 time units
4(b) Max of y1 on [0, 20] = +2.019886
     Min of y1 on [0, 20] = -2.019885
```

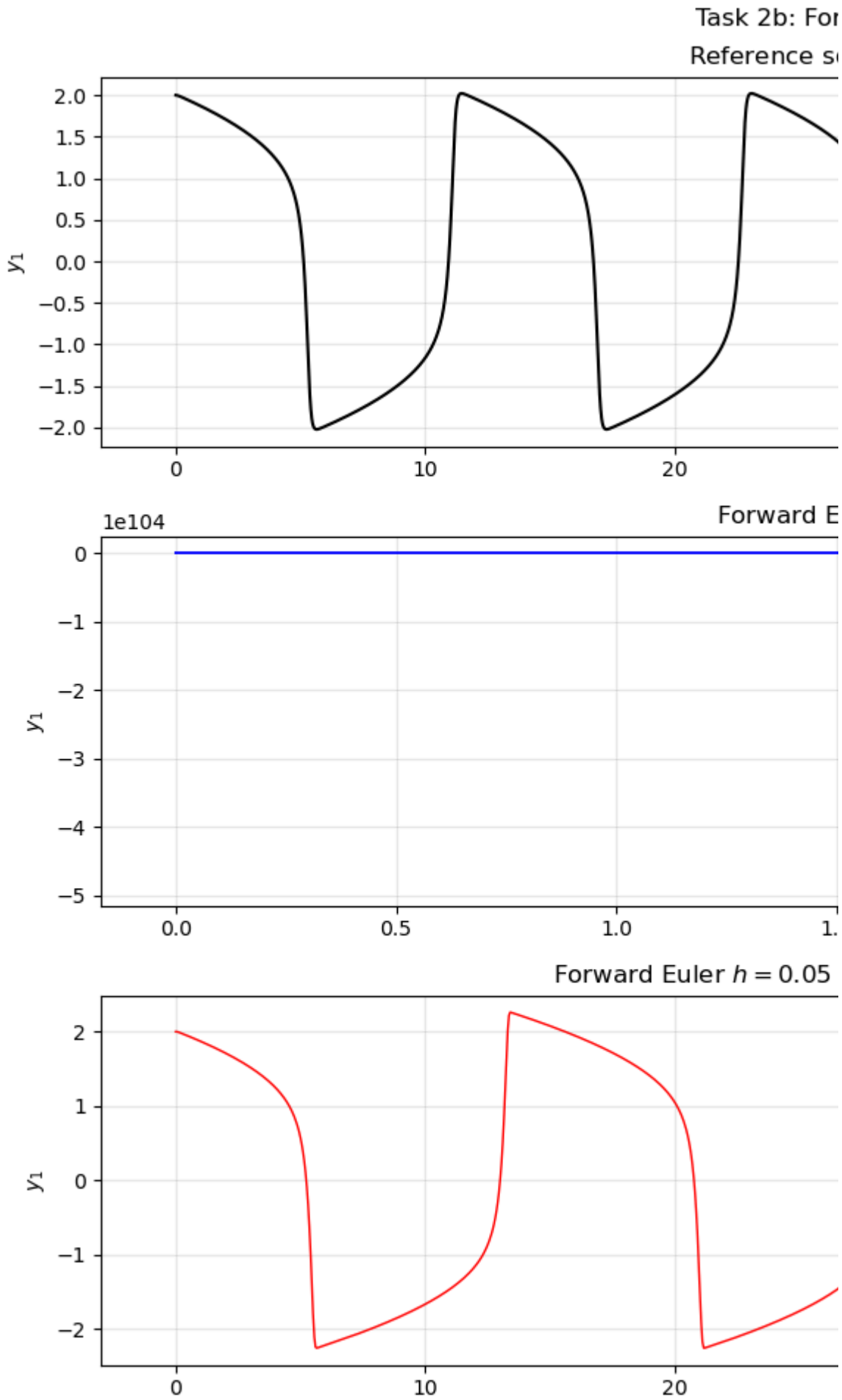
## Task 2b: Forward Euler numerical stability (5 marks)

With $\mu = 5$, and initial conditions as above, use forward Euler with $h = 0.2$ and $h = 0.05$ on $t \in [0, 60]$. Also compute a reference solution using `solve_ivp`. Plot all three solutions separately and comment on what you observe.

In [8]:

```
mu = 5.0
y0 = np.array([2.0, 0.0])
h_large = 0.2
h_small = 0.05
tf = 60.0

# implement forward Euler for Van der Pol with both step sizes
def euler_forward_system(T, f, y0, mu):
    """
    Forward Euler method for a system of ODEs.
    Parameters:
        T  : time grid
        f  : RHS function f(t, y, mu)
        y0 : initial vector
        mu : parameter
    Returns:
        Y : array of shape (N, len(y0)) containing solution
    """
    N = len(T)
    Y = np.zeros((N, len(y0)))
    Y[0] = y0.copy()
    # suppress overflow warnings (we detect instability manually)
    with warnings.catch_warnings():
        warnings.simplefilter('ignore', RuntimeWarning)
        for i in range(N - 1):
            h = T[i + 1] - T[i]
            Y[i + 1] = Y[i] + h * f(T[i], Y[i], mu)
            # detect numerical blow-up
            if not np.all(np.isfinite(Y[i + 1])):
                Y[i + 1:] = np.nan
                break
    return Y

T_large = np.arange(0.0, tf + h_large * 0.5, h_large)
T_small = np.arange(0.0, tf + h_small * 0.5, h_small)
Y_large = euler_forward_system(T_large, vdp_eqs, y0, mu)
Y_small = euler_forward_system(T_small, vdp_eqs, y0, mu)

sol_ref = solve_ivp(lambda t, y: vdp_eqs(t, y, mu), [0.0, tf], y0,
                    rtol=1e-8, atol=1e-10, dense_output=True)
t_ref = np.linspace(0.0, tf, 6000)
y_ref = sol_ref.sol(t_ref)

fin_idx      = np.where(~np.isfinite(Y_large[:, 0]))[0]
blow_up_step = fin_idx[0] if len(fin_idx) > 0 else len(T_large)

# plot all solutions
fig, axes = plt.subplots(3, 1, figsize=(12, 10))
axes[0].plot(t_ref, y_ref[0], 'k-', lw=1.5)
axes[0].set(ylabel='$y_1$', title=f'Reference solution (solve_ivp, $\\mu={mu}$)');
axes[0].grid(True, alpha=0.3)
axes[1].plot(T_large[:blow_up_step], Y_large[:blow_up_step, 0], 'b-', lw=1.3)
axes[1].axvline(T_large[blow_up_step-1], color='red', ls='--', lw=1.5,
                label=f'Blows up at $t \\approx {T_large[blow_up_step-1]:.2f}$')
axes[1].set(ylabel='$y_1$', title=f'Forward Euler $h={h_large}$ - UNSTABLE')
```

```
axes[1].legend(loc='upper right'); axes[1].grid(True, alpha=0.3)
axes[2].plot(T_small, Y_small[:, 0], 'r-', lw=1.0)
axes[2].set(xlabel='$t$', ylabel='$y_1$', title=f'Forward Euler $h={h_small}$ - stable but accumulates phase error')
axes[2].grid(True, alpha=0.3)
plt.suptitle('Task 2b: Forward Euler stability', fontsize=12)
plt.tight_layout(); plt.show()

print(f'h={h_large}: blows up at step {blow_up_step}  (t = {T_large[blow_up_step-1]:.3f})')
print(f'h={h_small}: all finite = {np.all(np.isfinite(Y_small))}')
```




h=0.2: blows up at step 18  (t = 3.400)
h=0.05: all finite = True

**Comment:**

The Van der Pol oscillator with $\mu = 5$ is moderately stiff. During the sharp transients of the limit cycle the Jacobian has a large negative eigenvalue $\sim O(\mu^2)$, imposing a stability constraint $h \lesssim 2/\mu^2 = 0.08$ on explicit methods.

- $h = 0.2$: Forward Euler is **numerically unstable** — The step size is too large to resolve the fast transient behaviour, so errors grow rapidly and the solution blows up.
- $h = 0.05$: Forward Euler is **numerically stable** ($0.05 < 0.08$) - The step size is smaller so much more stable. However, although it does not blow up, the solution gradually drifts out of phase compared to the reference solution. This is due to the fact that forward Euler has only first-order accuracy, so errors accumulate over time.
- **Reference (solve_ivp):** The adaptive RK45 solver adjusts its step size through each stiff transient, which allows it to remain both stable and accurate, faithfully tracing the limit cycle throughout $[0, 60]$. This highlights the limitations of fixed-step explicit methods for moderately stiff problems.

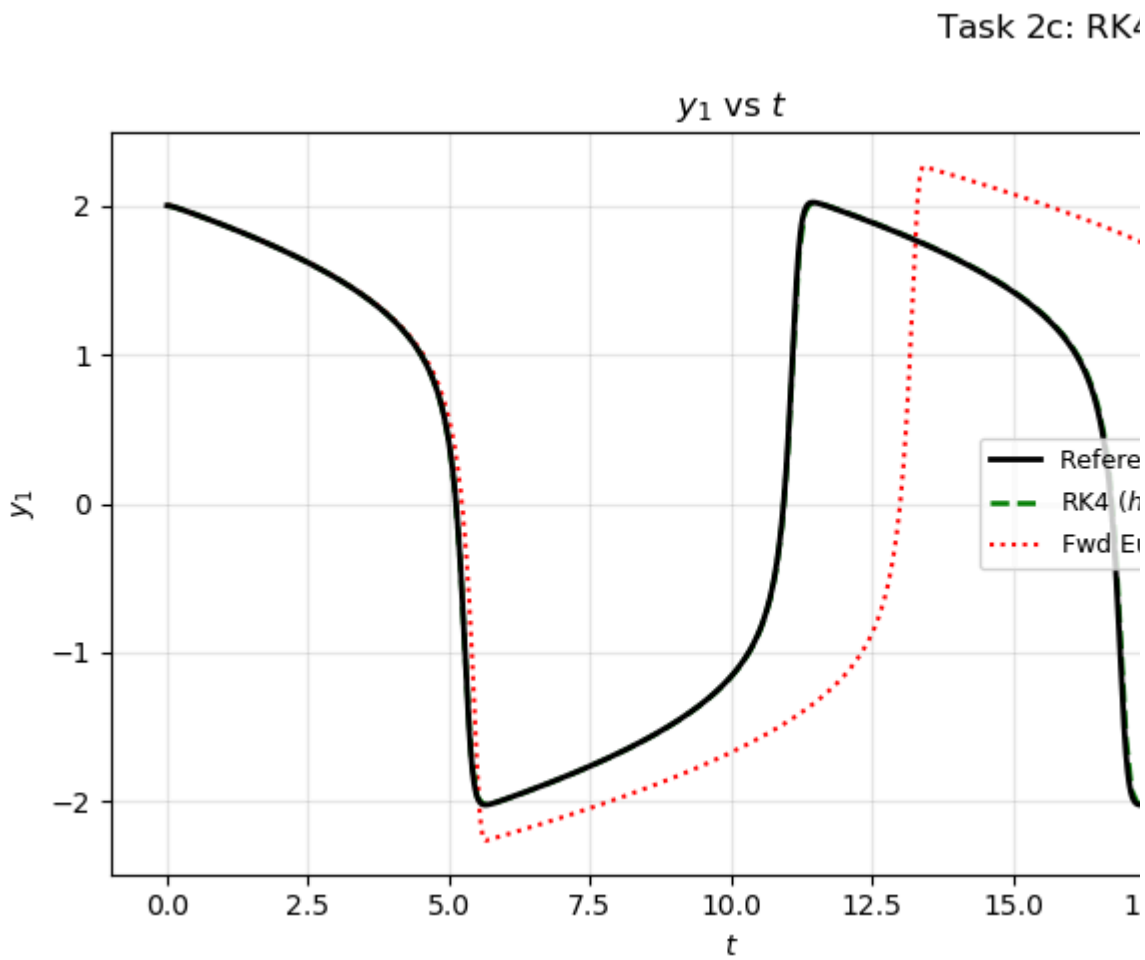
## Task 2c: RK4 comparison (7 marks)

With $\mu$ and initial conditions as in Task 2b, compare forward Euler (with $h = 0.05$) and RK4 (with $h = 0.1$) on $t \in [0, 20]$. Use `solve_ivp` with tight tolerances (`rtol=1e-9`, `atol=1e-11`) as a reference solution. Plot all three solutions on the same axes (both $y_1$ vs $t$ and $y_2$ vs $t$). Discuss the performance of both forward Euler and RK4.

In [9]:

```
mu = 5.0
y0 = np.array([2.0, 0.0])
tf = 20.0

# implement RK4 with h=0.1, forward Euler with h=0.05, and solve_ivp reference
# Classical fourth-order Runge-Kutta method (RK4) for a system of ODEs.
def rk4_system(T, f, y0, mu):
    N = len(T)
    Y = np.zeros((N, len(y0)))
    Y[0] = y0.copy()
    for i in range(N - 1):
        h  = T[i + 1] - T[i]
        k1 = f(T[i],        Y[i],           mu)
        k2 = f(T[i] + 0.5*h,  Y[i] + 0.5*h*k1,  mu)
        k3 = f(T[i] + 0.5*h,  Y[i] + 0.5*h*k2,  mu)
        k4 = f(T[i] + h,      Y[i] + h*k3,      mu)
        # update formula
        Y[i + 1] = Y[i] + (h / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
    return Y

sol_ref = solve_ivp(lambda t, y: vdp_eqs(t, y, mu), [0.0, tf], y0,
```

```
                rtol=1e-9, atol=1e-11, dense_output=True)
t_ref = np.linspace(0.0, tf, 4000)
y_ref = sol_ref.sol(t_ref)

T_fe   = np.arange(0.0, tf + 0.025, 0.05)
T_rk4  = np.arange(0.0, tf + 0.05,  0.1)
Y_fe   = euler_forward_system(T_fe,  vdp_eqs, y0, mu)
Y_rk4  = rk4_system(T_rk4, vdp_eqs, y0, mu)

# plot results
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, comp, lbl in zip(axes, [0, 1], ['$y_1$', '$y_2$']):
    ax.plot(t_ref,  y_ref[comp],    'k-',  lw=2.0, zorder=4, label='Reference (solve_ivp)')
    ax.plot(T_rk4,  Y_rk4[:, comp], 'g--', lw=1.8, zorder=3, label='RK4 ($h=0.1$)')
    ax.plot(T_fe,   Y_fe[:,  comp], 'r:',  lw=1.5, zorder=2, label='Fwd Euler ($h=0.05$)')
    ax.set(xlabel='$t$', ylabel=lbl, title=f'{lbl} vs $t$')
    ax.legend(fontsize=9); ax.grid(True, alpha=0.3)
plt.suptitle(f'Task 2c: RK4 vs Forward Euler vs Reference ($\\mu={mu}$)',
fontsize=12)
plt.tight_layout(); plt.show()

err_fe  = np.max(np.linalg.norm(Y_fe  - sol_ref.sol(T_fe).T,  axis=1))
err_rk4 = np.max(np.linalg.norm(Y_rk4 - sol_ref.sol(T_rk4).T, axis=1))
print(f'Max global error -- Forward Euler  (h=0.05) : {err_fe:.4e}')
print(f'Max global error -- RK4            (h=0.1)  : {err_rk4:.4e}')
print(f'RK4 is ~{err_fe/err_rk4:.0f}x more accurate despite using 2x the step size')
```




```
Max global error -- Forward Euler  (h=0.05) : 8.6206e+00
Max global error -- RK4            (h=0.1)  : 9.8778e-01
```

RK4 is ~9x more accurate despite using 2x the step size

**Discussion:**

The difference between the two methods is very clear from the plots and error values.

Even though RK4 uses a larger step size ($h = 0.1$ compared to $h = 0.05$ for forward Euler), it tracks the reference solution much more closely.

This makes sense because RK4 is fourth-order accurate, meaning the global error behaves like

$$O(h^4),$$

while forward Euler behaves like

$$O(h).$$

Forward Euler remains stable in this case, but it accumulates noticeable phase error over time. RK4, on the other hand, preserves both amplitude and phase much more accurately.

So despite using fewer steps, RK4 is significantly more efficient and accurate for this problem.

## Task 2d: Error vs step size (4 marks)

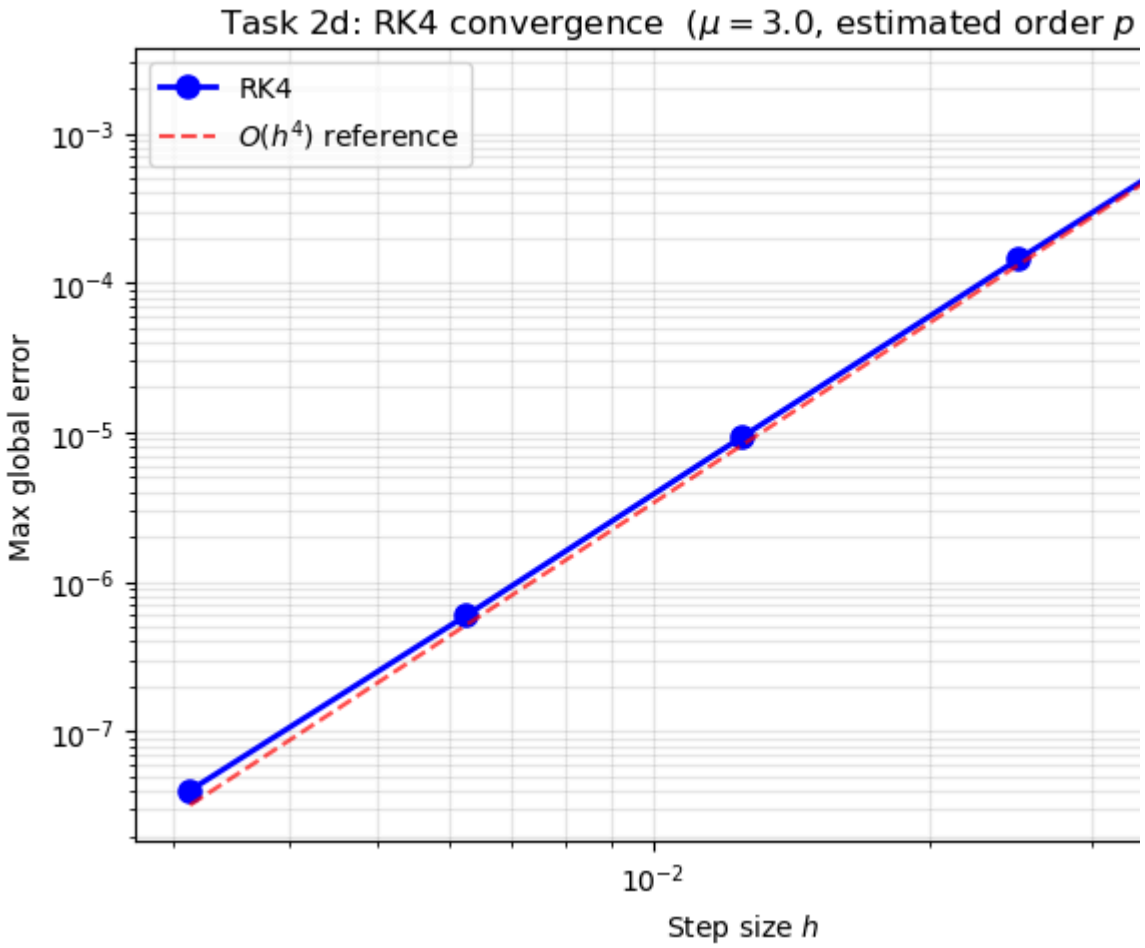
With $\mu$ and initial conditions as in Task 2b, use RK4 with step sizes $h \in \{0.05(\frac{1}{2})^m : m \in \{0, 1, 2, 3, 4\}\}$ on $t \in [0, 10]$. Compute the maximum global error norm over the interval against a reference solution from `solve_ivp` with tight tolerances: $\max_{n=0}^{N} \|y_n - y_{\text{ref}}(t_n)\|$. Create a log-log plot of error vs $h$ and estimate the observed order of convergence by fitting a line. You can use the `np.polyfit` function or any other reasonable method to fit the line.

In [10]:

```
mu = 3.0
y0 = np.array([2.0, 0.0])
tf = 10.0
h = 0.05 * 0.5 ** np.arange(5)

# implement RK4 and compute reference solution
sol_ref = solve_ivp(lambda t, y: vdp_eqs(t, y, mu), [0.0, tf], y0,
                    rtol=1e-10, atol=1e-12, dense_output=True)

# compute maximum global error over time for each h
max_errors = []
```

```python
for h_val in h:
    T = np.arange(0.0, tf + h_val * 0.5, h_val)
    T = T[T <= tf + 1e-12]
    Y     = rk4_system(T, vdp_eqs, y0, mu)
    Y_ref = sol_ref.sol(T).T
    max_errors.append(np.max(np.linalg.norm(Y - Y_ref, axis=1)))

max_errors = np.array(max_errors)

# estimate order p using np.polyfit on log-log data
p = np.polyfit(np.log(h), np.log(max_errors), 1)[0]

print(f"{'h':>10}  {'Max Error':>14}  {'Ratio (expected ~16 for RK4)':>30}")
print('-' * 58)
for i, h_val in enumerate(h):
    ratio = max_errors[i-1] / max_errors[i] if i > 0 else float('nan')
    print(f'{h_val:>10.6f}  {max_errors[i]:>14.6e}  {ratio:>30.4f}')
print(f'\nEstimated order of convergence (np.polyfit): p = {p:.3f}  (expected ~4 for RK4)')

# plot log-log
plt.figure(figsize=(7, 5))
plt.loglog(h, max_errors, 'bo-', lw=2, ms=8, label='RK4')
h_line = np.array([h.min(), h.max()])
plt.loglog(h_line, max_errors[0] * (h_line / h[0])**4, 'r--', alpha=0.7, label=r'$O(h^4)$ reference')
plt.xlabel('Step size $h$')
plt.ylabel(r'Max global error')
plt.title(f'Task 2d: RK4 convergence  ($\\mu={mu}$, estimated order $p={p:.2f}$)')
plt.legend(); plt.grid(True, alpha=0.3, which='both'); plt.tight_layout(); plt.show()
```

```
         h      Max Error    Ratio (expected ~16 for RK4)
----------------------------------------------------------
  0.050000   2.110769e-03                             nan
  0.025000   1.441904e-04                         14.6388
  0.012500   9.362586e-06                         15.4007
  0.006250   5.967774e-07                         15.6886
  0.003125   4.035011e-08                         14.7900

Estimated order of convergence (np.polyfit): p = 3.927  (expected ~4 for RK4)
```



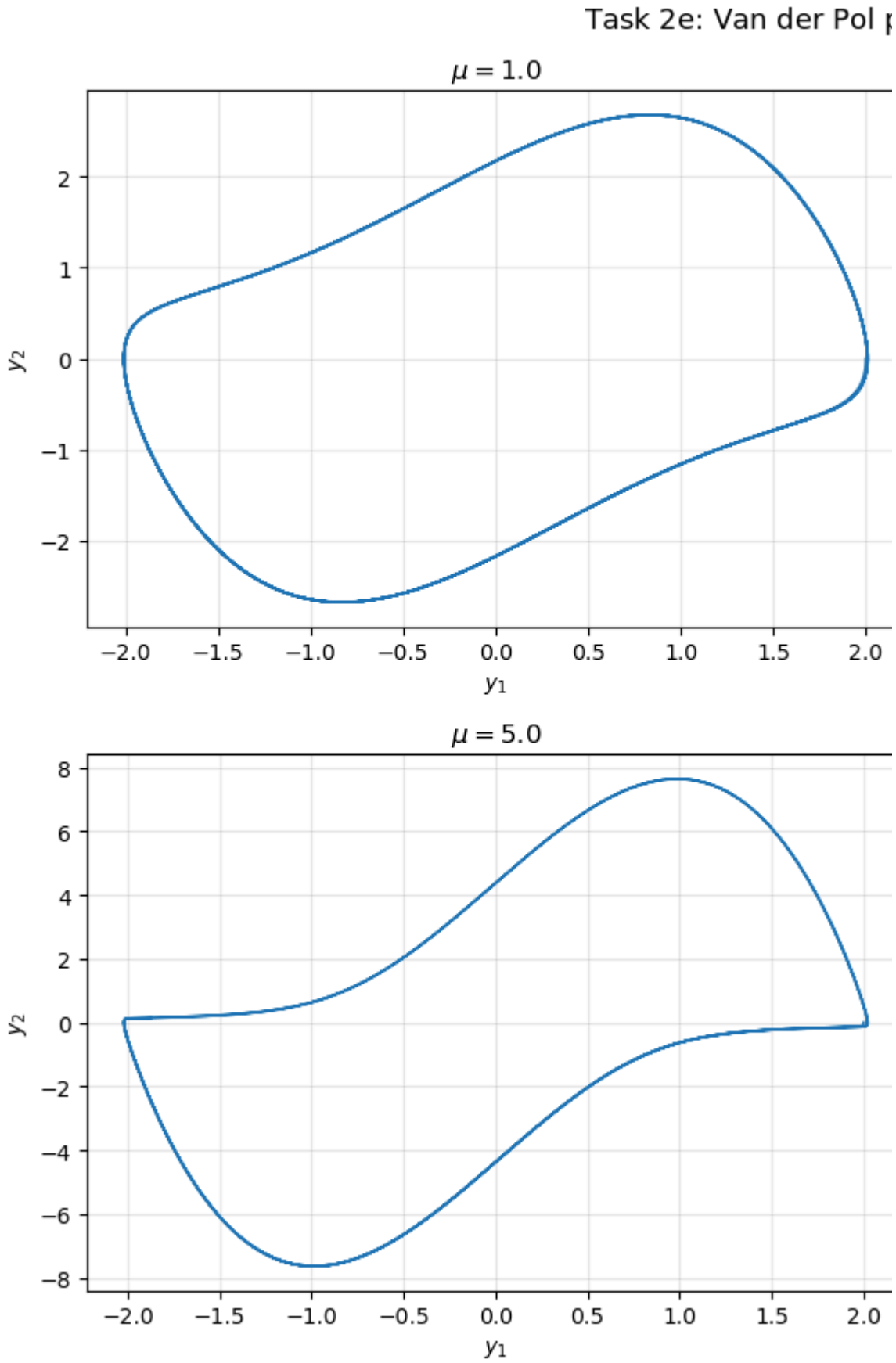

## Task 2e: Varying stiffness parameter (3 marks)

With initial conditions as in Task 2b, simulate with `solve_ivp` for $\mu \in \{1, 2, 5, 10\}$ and $t \in [0, 30]$. For each $\mu$, produce a phase-plane plot ($y_1$ vs $y_2$). Describe the qualititative changes you see in solutions as $\mu$ is varied.

In [11]:

```
mu_values = [1.0, 2.0, 5.0, 10.0]
y0 = np.array([2.0, 0.0])
tf = 30.0

# loop over mu_values
fig, axes = plt.subplots(2, 2, figsize=(12, 9))
axes = axes.flatten()

# solve with solve_ivp
for ax, mu in zip(axes, mu_values):
    sol = solve_ivp(lambda t, y, m=mu: vdp_eqs(t, y, m), [0.0, tf], y0,
                    rtol=1e-8, atol=1e-10, dense_output=True)
    # plot results
    t_plot = np.linspace(0.0, tf, 6000)
    y_plot = sol.sol(t_plot)
    ax.plot(y_plot[0], y_plot[1], lw=1.2)
    ax.set(xlabel='$y_1$', ylabel='$y_2$', title=f'$\\mu = {mu}$')
    ax.grid(True, alpha=0.3)

plt.suptitle(r'Task 2e: Van der Pol phase planes for varying $\mu$', fontsize=13)
```

```
plt.tight_layout()
plt.show()
```




**Description of qualitative changes:**

As $\mu$ increases, the qualitative behaviour of the oscillator changes significantly.

- $\mu = 1$: The limit cycle is **nearly circular/elliptical**. The oscillation is almost sinusoidal — nonlinear damping is mild. The equation is not stiff.
- $\mu = 2$: The limit cycle is **visibly more distorted** from the ellipse, with faster dynamics in some phases.
- $\mu = 5$: The limit cycle has **sharp corners** and **flat sides**, indicating **relaxation oscillation** — long slow drift phases interrupted by rapid near-vertical jumps. The equation is visibly stiff.
- $\mu = 10$: The phase portrait is **nearly rectangular**, it has fully achieved **relaxation oscillation** — The solution moves slowly along two nearly horizontal branches and then switches very quickly between them. The stiffness is severe — fast transients occur on timescale $O(1/\mu^2) = 0.01$, orders of magnitude shorter than the oscillation period $O(\mu) \approx 10$.

So increasing $\mu$ makes the system more nonlinear and more stiff, which explains why explicit methods struggle more as $\mu$ grows.

In [ ]:

```
```